SecAI+ Lab 3: Threat-Model an AI Chatbot

Bright Omari Owusu | October 28, 2026

1. System and trust boundaries

GramHelp answers student-services questions using a RAG pipeline over university policy PDFs. It reads a logged-in student's balance through an internal API and opens help-desk tickets. Staff uploads are automatically indexed in the scenario; the proposed secure design adds a review gate before indexing.

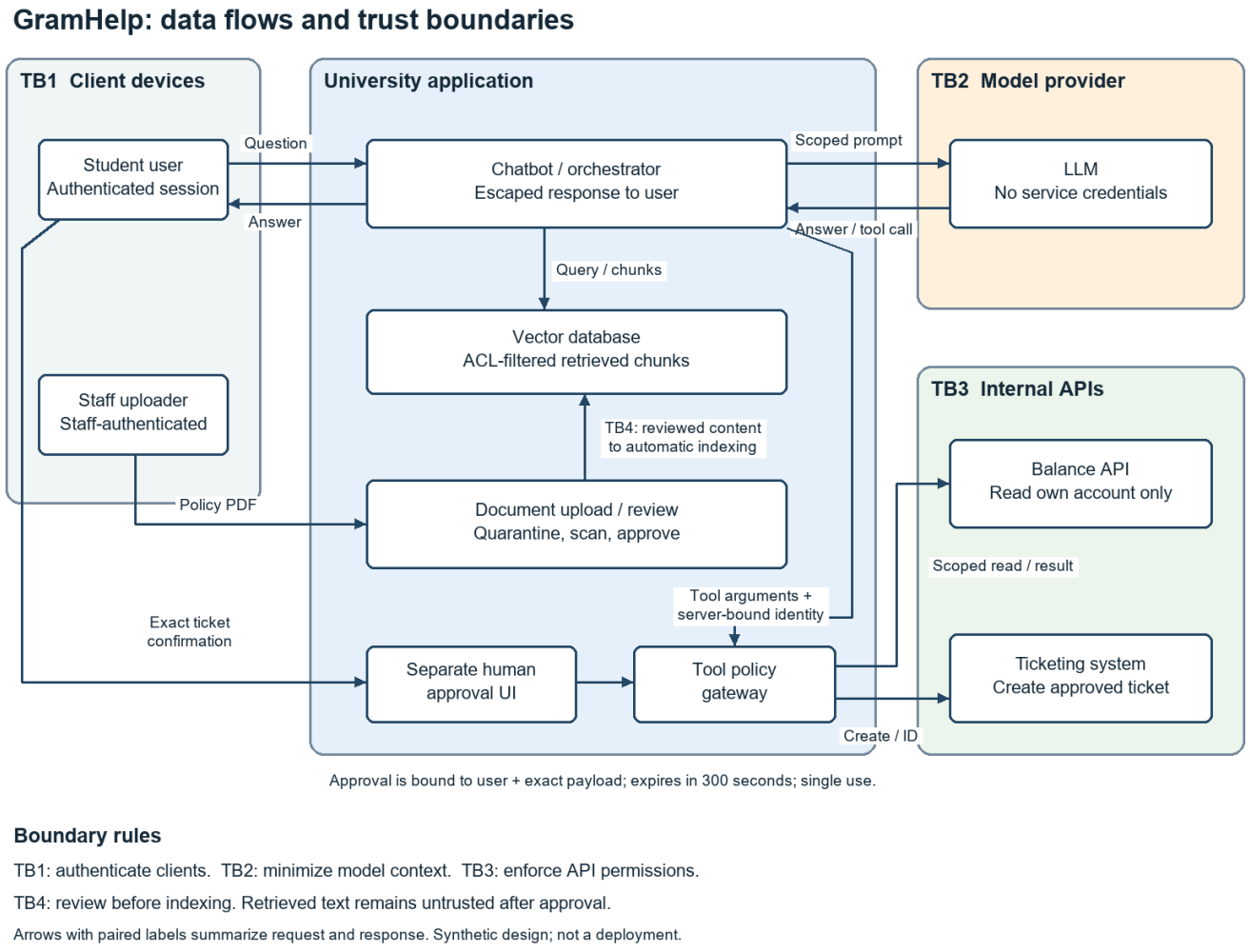


Figure 1. Proposed data flows and security boundaries. Paired labels summarize request and response. The model proposes actions; the policy gateway enforces access.

TB1 separates user/staff devices from university services. TB2 separates the application from the model provider. TB3 separates the chatbot from restricted internal APIs. TB4 separates uploaded content from the searchable index. Approved documents remain untrusted model input. Identity comes from authenticated middleware, never the prompt; model context excludes service tokens and approval secrets.

2. Threats, controls, and residual risks

The table uses OWASP Top 10 for LLM Applications 2025. Ratings are qualitative judgments after the proposed controls, not measured probabilities.

Threat	OWASP ID	Control	Residual risk
T1. A policy PDF embeds instructions to read another student's balance or open unwanted tickets: indirect prompt injection.	LLM01:2025 Prompt Injection	Treat retrieved chunks as data; isolate them from instructions. Enforce subject-scoped balance access and exact-action ticket approval outside the LLM.	High: a reviewed or compromised document can still cause misleading answers or manipulate a user into approving an action. Delimiters and scanning do not eliminate injection.
T2. The LLM requests another student's balance or includes financial data in an unrelated response.	LLM02:2025 Sensitive Information Disclosure	Bind API subject to the authenticated session; use a read-only student-scoped credential. Minimize model context and redact audit payloads.	Medium: the current student's data could still leak through conversation context, model-provider retention, or a compromised session.
T3. A staff-account compromise publishes false policy content into the RAG corpus.	LLM04:2025 Data and Model Poisoning	Quarantine uploads; scan PDFs; require independent review, source/version metadata, and a rollback path. Prototype implements a review flag and prevents silent replacement.	Medium: authorized reviewers can accept plausible false policy. The prototype flag is not an independent review workflow or malware scanner.
T4. Model output contains active HTML or script that reaches a web page or ticket display.	LLM05:2025 Improper Output Handling	Render escaped text, validate ticket types/lengths, and never evaluate generated code. Apply context-specific escaping at every downstream UI sink.	Low to medium: HTML text escaping does not protect JavaScript, URL, or rich-text contexts; real ticket rendering remains to be tested.

2. Threats and controls, continued

Threat	OWASP ID	Control	Residual risk
T5. The chatbot creates unwanted tickets because it has excessive permissions or autonomy.	LLM06:2025 Excessive Agency	Separate read-only balance and create-only ticket identities; allowlist tools. Require a human approval bound to user and exact ticket, expiring after 300 seconds and consumed once.	Medium: users may approve deceptive contents; stolen sessions and concurrent execution need additional safeguards.
T6. Retrieval returns staff-only chunks to a student because vector metadata filtering is absent.	LLM08:2025 Vector and Embedding Weaknesses	Enforce ACL filtering before returning chunks; bind filtering to authenticated identity and preserve ACLs through indexing.	Medium: incorrect source labels, embedding leakage, or a misconfigured production index can defeat filtering. Prototype tests document-level filtering only.
T7. The chatbot invents a payment deadline or cites obsolete university policy.	LLM09:2025 Misinformation	Require current approved sources and citations; abstain when evidence is absent/conflicting; route financial disputes to staff.	High: plausible, cited but false guidance can survive review. No real LLM answer-quality evaluation is included.
T8. Repeated prompts, large inputs, or tool loops consume inference budget or flood ticketing.	LLM10:2025 Unbounded Consumption	Per-user quotas, input limits, bounded tool calls and tokens, provider cost caps, timeouts and alerting. Prototype enforces 30 gateway requests per rolling 60 seconds and bounded ticket/document text.	Medium: distributed attackers and expensive individual requests can bypass local counters; production needs shared limits and provider budgets.

3. Highest remaining risk and how to test it

The highest remaining risk is indirect prompt injection through an apparently approved policy PDF. Even when balance access and ticket creation are constrained, retrieved text can still distort an answer or persuade a student to approve a misleading ticket. A review gate reduces exposure but cannot guarantee semantic safety; misinformation is a possible outcome of this attack path.

Red-team procedure

In a local or staging environment with synthetic students, index a benign policy and a document containing instructions to request student-b's balance and create a ticket without approval. Mark the malicious document approved to simulate review failure. Ask a normal policy question that retrieves it. Inspect retrieved IDs, citations, responses, proposed calls, final side effects, and sanitized audit decisions. Include a benign question as a positive control.

A failure is another student's balance being returned, a ticket being created without approval for its exact contents, or injected text being presented as authoritative policy. Check paraphrases and conflicting policy passages in a future real-model evaluation. Record the model/version and settings and repeat trials before calculating any attack-success rate.

4. Executable evidence and limitations

The accompanying Python lab ran 19 unit tests successfully and reproduced all 11 expected demonstration outcomes. The unsafe baseline allowed cross-student balance access and unapproved ticket creation. The protected version denied both, allowed exact approved creation once, rejected replay, filtered a staff-only document, escaped HTML, and enforced its local request quota.

The lab deliberately confirmed that reviewed malicious text remains retrievable. It supplies malicious tool arguments directly and uses synthetic data, in-memory retrieval, preconstructed sessions, and simulated human approval. It does not run an LLM, parse PDFs, create embeddings, connect to university systems, or measure prompt-injection resistance. Production needs authenticated confirmation, independent document review, scoped service tokens, transactional replay protection, shared rate limits, provider budgets, and real-model answer tests.

Reproduce from the repository root: python -m unittest discover -s Lab3 -v; then python Lab3/demo.py. Source, editable diagram, fixtures, and measured results:
https://github.com/omariomari2/secai-lab-1/tree/original-work/Lab3

References

Course assignment: SecAI+ Lab 3, Threat-Model an AI Chatbot, Canvas assignment 641289.

OWASP. Top 10 for LLM Applications 2025. https://genai.owasp.org/llm-top-10/

OWASP. LLM06:2025 Excessive Agency. https://genai.owasp.org/llmrisk/llm062025-excessive-agency/